

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

Направленность (профиль)
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка казуальной компьютерной игры и анализ ее продвижения на
платформе Яндекс.Игры

Обучающегося 4 курса
очной формы обучения
Зубова Михаила Геннадьевича

Руководитель выпускной
квалификационной работы:
заведующий кафедрой ИТиЭО
доктор педагогических наук
Власова Елена Зотиковна

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ И ПРОДВИЖЕНИЯ КАЗУАЛЬНЫХ КОМПЬЮТЕРНЫХ ИГР	5
1.1 Современное состояние рынка казуальных компьютерных игр.....	5
1.2 Казуальные игры: определение, классификация и ключевые особенности ...	7
1.3 Жанр rogue-lite и его влияние на современный геймдизайн	9
1.4 Платформа Яндекс.Игры: технические требования, аудитория и инструменты продвижения	11
1.5 Сравнительный анализ инструментов разработки	12
1.6 Обзор дополнительных технологий: архитектурные подходы, SDK и паттерны проектирования	14
1.7 Обоснование выбора средств разработки и архитектурного подхода	16
ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ ПРОДВИЖЕНИЯ ИГРОВОГО ПРИЛОЖЕНИЯ	18
2.1 Постановка задачи на проектирование и разработку	18
2.2 Архитектура игрового приложения	21
2.3 Реализация ключевых компонентов.....	25
2.4 Тестирование и оптимизация.....	29
2.5 Анализ механизмов продвижения на платформе Яндекс.Игры.....	32
2.6 Итоги практической реализации и анализа	35
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ	39

ВВЕДЕНИЕ

Индустрия компьютерных игр на протяжении последних лет демонстрирует устойчивый рост, а расширение аудитории мобильных и браузерных платформ способствует популяризации казуального сегмента. Современные игровые платформы, в частности Яндекс.Игры, предоставляют разработчикам не только среду для публикации продукта, но и встроенные инструменты монетизации, аналитики и продвижения. Благодаря этому малые команды и независимые разработчики получают возможность создавать конкурентоспособные проекты без необходимости выстраивать собственную дистрибуцию с нуля. Однако успех такой игры во многом определяется не только качеством её реализации, но и грамотным анализом механизмов продвижения на целевой платформе.

Казуальные игры отличаются низким порогом входа, простыми правилами и короткими игровыми сессиями, что делает их востребованными среди широкой аудитории. При этом разработка даже простой с точки зрения механик игры требует продуманного архитектурного подхода, использования современных игровых движков и соблюдения принципов модульности, расширяемости и читаемости кода. Параллельно с этим перед разработчиком встаёт задача изучения специфики конкретной платформы: её технических требований, правил размещения, алгоритмов ранжирования и инструментов взаимодействия с аудиторией. Данное противоречие между потребностью в качественном казуальном продукте, способном удерживать внимание пользователя, и недостаточной проработкой вопросов его проектирования и продвижения в рамках одной выбранной платформы обуславливает актуальность данной работы.

Целью дипломной работы является разработка казуальной компьютерной игры и анализ её продвижения на платформе Яндекс.Игры.

Для достижения поставленной цели были выделены следующие задачи:

- провести анализ современного состояния рынка казуальных компьютерных игр, выявить особенности жанра и ключевые факторы, влияющие на успешность продукта на платформе Яндекс.Игры;
- осуществить отбор инструментальных средств разработки, включая игровой движок, архитектурные подходы и дополнительные SDK;
- спроектировать архитектуру игрового приложения, разработать его ключевые компоненты и реализовать игровой цикл в соответствии с выбранной моделью;
- провести анализ механизмов продвижения на платформе Яндекс.Игры, оценить их эффективность применительно к разработанному продукту и сформулировать рекомендации по улучшению его видимости.

Объектом исследования является процесс разработки и продвижения казуальных компьютерных игр на веб-ориентированных игровых платформах.

Предметом исследования выступают инструменты и методы проектирования казуальной компьютерной игры на базе современного игрового движка, а также механизмы продвижения игрового продукта на отечественной платформе Яндекс.Игры.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ И ПРОДВИЖЕНИЯ КАЗУАЛЬНЫХ КОМПЬЮТЕРНЫХ ИГР

1.1 Современное состояние рынка казуальных компьютерных игр

Рынок компьютерных игр на протяжении последнего десятилетия демонстрирует устойчивый рост, при этом значительную долю как по объёму выручки, так и по численности аудитории занимает сегмент казуальных игр. Под казуальными (от англ. casual – повседневный) понимают игры, ориентированные на массовую аудиторию, обладающие простыми правилами, интуитивным управлением и короткими игровыми сессиями, что позволяет вовлекать пользователей, не имеющих большого игрового опыта [1, с.12–15]. В отличие от сложных и комплексных игровых проектов, требующих значительных временных затрат и специальных навыков, казуальные игры рассчитаны на эпизодическое взаимодействие, часто в промежутках между повседневными делами.

По данным аналитических отчётов, в 2023–2024 годах казуальный сегмент продолжал расширяться, чему способствовало несколько факторов. Одним из ключевых факторов расширения казуального сегмента стало широкое распространение мобильных устройств и повышение качества интернет-соединений, благодаря чему игры стали доступны сотням миллионов пользователей [2]. Параллельно с этим появились специализированные платформы, такие как Яндекс.Игры, которые предоставляют разработчикам готовую инфраструктуру для публикации, монетизации и продвижения продуктов, снимая необходимость выстраивать собственные каналы дистрибуции [3]. Ещё одним драйвером роста выступила модель free-to-play с внутриигровыми покупками или рекламой, доказавшая свою экономическую эффективность именно в казуальном сегменте: низкий порог входа способствует быстрому накоплению массовой аудитории [20].

Отдельного внимания заслуживают браузерные игры, работающие непосредственно в веб-окружении и не требующие установки дополнительного программного обеспечения. Для начала игровой сессии пользователю достаточно перейти по ссылке, что сводит к минимуму барьеры для вовлечения. Запущенная в 2020 году платформа Яндекс.Игры агрегирует подобные проекты и предлагает разработчикам инструменты аналитики, рекламной монетизации и взаимодействия с аудиторией, делая её привлекательной средой для независимых студий и индивидуальных разработчиков [3]. По данным официальной аналитики платформы, её ежемесячная аудитория составляет десятки миллионов активных пользователей, а каталог насчитывает десятки тысяч игр различных жанров [11].

Наблюдаемые рыночные тенденции указывают на заметный сдвиг внутри казуального сегмента в сторону игр, сочетающих повторяющиеся короткие сессии с необратимой потерей прогресса при поражении и внутриигровой прогрессией. Подобные механики, оставаясь простыми в освоении, создают вариативный игровой опыт, который стимулирует повторные прохождения. Ряд авторов относят такие гибридные формы к поджанру *rogue-lite* и подчёркивают, что в последние годы они привлекают значительную аудиторию и демонстрируют высокие показатели удержания [13, с.45–48].

Таким образом, современный рынок казуальных игр характеризуется высокой конкуренцией, но одновременно предоставляет широкие возможности для новых проектов, особенно при условии грамотного использования платформенных инструментов продвижения и учёта актуальных игровых трендов. Данные обстоятельства делают актуальным как проектирование конкурентоспособного казуального продукта, так и анализ механизмов его продвижения на конкретной платформе, что и составляет предмет настоящей работы.

1.2 Казуальные игры: определение, классификация и ключевые особенности

Термин «казуальная игра» (от англ. casual – повседневный) прочно вошёл в обиход как разработчиков, так и исследователей, однако его содержательное наполнение остаётся предметом дискуссий. Ряд авторов трактует казуальность прежде всего через простоту правил и управления, позволяющую вовлекать пользователей без специальной подготовки [1, с.12–15]. Другие исследователи расширяют это понимание, включая в него временной аспект: продолжительность игровой сессии должна быть достаточно малой, чтобы продукт мог потребляться фрагментарно, в промежутках между повседневными занятиями [10, с.112–114]. Третья точка зрения акцентирует социально-психологическую функцию казуальных игр – они выступают средством кратковременного отдыха и эмоциональной разгрузки, а не источником длительного напряжения или соревновательного стресса [9, с.88–92]. Такое многообразие подходов свидетельствует о том, что казуальная игра представляет собой не жёстко очерченный жанр, а скорее набор характеристик, комбинация которых может варьироваться в зависимости от платформы, аудитории и сегмента рынка.

Обобщая существующие подходы, можно выделить следующие ключевые особенности, присущие большинству казуальных игр:

- Простота правил. Игровые механики осваиваются за считанные минуты, а управление часто сводится к одному-двум действиям (клик, свайп, перемещение) [1, с. 28].
- Короткая игровая сессия. Продолжительность одного раунда обычно не превышает нескольких минут, что делает игру удобной для фрагментарного потребления [10, с. 112].
- Низкие системные требования и широкая доступность. Казуальные проекты ориентированы на массовые устройства – смартфоны, планшеты, веб-браузеры – и не требуют мощного аппаратного обеспечения [3].

- Гибкая монетизация. Наибольшее распространение получили модели free-to-play с рекламными интеграциями или опциональными микроплатежами, позволяющие разработчику получать доход без взимания платы за вход [20].

В зависимости от платформы, целевой аудитории и преобладающих механик казуальные игры принято классифицировать на несколько групп. Наиболее многочисленными являются мобильные казуальные проекты, которые по состоянию на 2024 год формируют значительную долю дохода всего игрового рынка [2]. Отдельную нишу занимают браузерные казуальные игры, работающие непосредственно в веб-окружении и не требующие установки дополнительного программного обеспечения [3]. Внутри этих групп выделяют гиперказуальные игры – максимально упрощённые, часто с одной механикой – головоломки, кликеры, «time-killer» и гибридные формы, заимствующие элементы из других жанров.

Особый интерес для настоящего исследования представляет тенденция к гибридизации, наблюдаемая в казуальном сегменте в последние годы. Проекты, объединяющие казуальную доступность с элементами жанра rogue-lite, демонстрируют высокие показатели вовлечённости и удержания аудитории. В таких играх короткие игровые сессии сочетаются с необратимой потерей прогресса при гибели персонажа и внутриигровой прогрессией, реализуемой через выбор улучшений в реальном времени [13, с.45–48]. Эта формула, популяризированная рядом успешных проектов, сформировала новый поджанр, который иногда обозначают как «Vampire Survivors-like» – по названию игры, сделавшей подобный геймдизайн широко известным. Детальное рассмотрение данного поджанра и его ключевых механик будет проведено в следующем параграфе.

Таким образом, казуальные игры представляют собой динамично развивающийся сегмент, характеризующийся высоким разнообразием форм и быстрым заимствованием механик из различных жанров. Понимание их базовых

особенностей и классификаций необходимо для осмысленного проектирования конкурентоспособного продукта.

1.3 Жанр rogue-lite и его влияние на современный геймдизайн

Термин rogue-lite возник как производный от roguelike – поджанра компьютерных ролевых игр, восходящего к классической игре Rogue (1980 г.) и характеризующегося процедурной генерацией уровней, пошаговым режимом и необратимой гибелью персонажа, обнуляющей весь достигнутый игроком прогресс [13, с.12–18]. В отличие от оригинальных roguelike, проекты категории rogue-lite заимствуют лишь отдельные игровые механики изначального жанра, смягчая их и комбинируя с элементами других жанров. Наиболее существенным отличием является наличие внутриигровой прогрессии, частично сохраняющейся между прохождениями либо активно развивающейся в рамках одной сессии, что снижает порог входа и делает игру более дружелюбной к массовой аудитории [13, с.45–48].

Ключевыми чертами, которые rogue-lite привнёс в современный геймдизайн и которые оказались особенно востребованными в казуальном сегменте, являются:

- Короткая, самодостаточная игровая сессия. Одна попытка прохождения (забег) длится от нескольких минут до получаса, после чего игрок, независимо от исхода, может начать новое. Это идеально согласуется с казуальным паттерном потребления, описанным в предыдущем параграфе [10, с.112–114].
- Необратимость неудачи. Гибель персонажа означает завершение попытки без возможности загрузки сохранения, что придаёт каждому действию значимость, но не наказывает игрока сверх меры, поскольку новое прохождение начинается быстро.
- Прогрессия в рамках одной игровой сессии. В ходе «забега» игрок получает ресурсы (опыт, улучшения), позволяющие наращивать силу

персонажа. Выбор между несколькими вариантами развития создаёт тактическую глубину без усложнения базовых правил.

- Простое, часто автоматизированное управление. Многие rogue-lite минимизируют количество действий игрока: например, персонаж атакует автоматически, а пользователь лишь выбирает направление движения и улучшения. Это снижает требования к навыкам и расширяет аудиторию [8, с.50–55].

Совокупность перечисленных свойств привела к формированию гибридных форм, находящихся на стыке казуального дизайна и rogue-lite. Наиболее ярким представителем такого синтеза стала игра «Vampire Survivors» (2022 г.), создавшая множество последователей и фактически выделившая поджанр, который неформально обозначают как Vampire Survivors-like [13, с.100–105]. В проектах этого типа игрок управляет только перемещением, а атаки выполняются автоматически с заданной периодичностью; основная стратегия сводится к выбору подходящих улучшений при повышении уровня и позиционированию в игровом пространстве.

Именно такая модель была выбрана в качестве ориентира для разрабатываемого в рамках текущей работы продукта. Она позволяет совместить казуальную доступность (одно действие – движение, короткая сессия, интуитивное управление) с глубиной, обеспечиваемой механиками rogue-lite (необратимость смерти, внутрисессионная прогрессия, выбор улучшений). Детальное проектирование и реализация этой модели будут раскрыты во второй главе.

Таким образом, жанр rogue-lite оказал значительное влияние на современный геймдизайн, предложив набор механик, которые одновременно удерживают внимание искушённой аудитории и остаются понятными для массового пользователя. Анализ данного жанра и его адаптация под конкретный проект являются необходимым шагом для обоснованного проектирования игрового продукта.

1.4 Платформа Яндекс.Игры: технические требования, аудитория и инструменты продвижения

Выбор платформы для публикации является одним из определяющих факторов, влияющих на архитектуру разрабатываемой игры, её монетизацию и стратегию продвижения. Для казуального продукта, ориентированного на массового пользователя и работающего в браузерном окружении, платформа Яндекс.Игры представляет особый интерес, поскольку сочетает широкую аудиторию, встроенные механизмы дистрибуции и развитый инструментарий для разработчика [3].

Платформа Яндекс.Игры представляет собой агрегатор браузерных игр, доступных без установки дополнительного программного обеспечения. Каталог платформы насчитывает десятки тысяч проектов различных жанров, а ежемесячная аудитория составляет десятки миллионов активных пользователей [11]. Ключевым преимуществом для разработчика является отсутствие необходимости самостоятельно решать задачи хостинга, аутентификации и приёма платежей – все эти функции берёт на себя платформа, что позволяет сосредоточиться на создании игрового контента [3].

С технической точки зрения Яндекс.Игры принимают проекты, реализованные в виде веб-приложений. Основным форматом поставки является WebGL-сборка, генерируемая многими современными игровыми движками. Платформа предъявляет ряд требований к публикуемым играм: ограничение на размер загружаемых файлов, обязательное использование HTTPS, совместимость с основными браузерами, а также интеграция Yandex Games SDK, предоставляющего доступ к функциям авторизации, рекламной монетизации, облачных сохранений и аналитики [14]. Соответствие этим требованиям гарантирует стабильную работу игры в веб-окружении и её доступность для аудитории платформы [15].

Аудитория Яндекс.Игр охватывает широкий круг пользователей, причём значительная доля посещений приходится на мобильные устройства [3]. Такой профиль аудитории хорошо согласуется с целевой группой казуальных игр: пользователи склонны к коротким игровым сессиям, не готовы тратить время на длительное обучение и ожидают немедленного вовлечения [10, с.112–114]. При этом внутри платформы существуют механизмы, позволяющие разработчику целенаправленно продвигать свой продукт.

Инструменты продвижения, предоставляемые Яндекс.Играми, можно условно разделить на несколько категорий. Платформа предоставляет разработчику несколько взаимосвязанных инструментов, влияющих на видимость игры. Прежде всего, действует внутренняя система ранжирования и рекомендаций: она опирается на анализ пользовательского поведения, и проекты с высокими показателями удержания и вовлечённости получают более заметные позиции в каталоге [11]. Дополнительно доступны рекламные инструменты, с помощью которых можно размещать оплаченные объявления как внутри самой платформы, так и в смежных сервисах Яндекса. Встроенные механизмы монетизации – показ рекламы или внутриигровые покупки – не только приносят доход, но и позволяют гибко выстраивать экономику продукта [20]. Наконец, аналитический кабинет разработчика даёт детальную картину поведения пользователей, источников трафика и эффективности рекламных кампаний, благодаря чему игру и стратегию её продвижения можно регулярно совершенствовать [11].

В совокупности эти средства образуют комплексную среду, которая, с одной стороны, упрощает техническую сторону публикации, а с другой – предоставляет встроенные механизмы для анализа и усиления видимости продукта.

1.5 Сравнительный анализ инструментов разработки

Выбор игрового движка является одним из ключевых решений, определяющих архитектуру будущего продукта, скорость разработки,

возможности публикации на целевых платформах и доступность инструментов для решения специфических задач. Для казуальной браузерной игры, ориентированной на платформу Яндекс.Игры, наиболее существенными критериями при сравнении движков выступают: поддержка двумерной графики и физики, возможность экспорта в WebGL, наличие языка программирования с низким порогом входа, качество документации и размер сообщества, лицензионная политика, а также совместимость с Yandex Games SDK. В таблице 1 представлено сопоставление трёх популярных игровых движков по перечисленным критериям.

Таблица 1.1 – Сравнение игровых движков для разработки WebGL-игры

Критерий	Unity	Unreal Engine 5	Godot 4
Язык программирования	C#	C++/ Blueprints	GScript / C++ / C#
2D-инструменты	Встроенная 2D-система	Ориентирован на 3D; 2D ограничено	Полноценный 2D-движок
Экспорт в WebGL	Полная поддержка	Поддержка WebGL, но сборки тяжеловесные	Экспорт в WebGL через HTML5
Порог входа	Средний	Высокий	Низкий
Документация и сообщество	Обширная документация; крупное сообщество	Профессиональная документация; крупное сообщество, но в AAA-сегменте	Документация постепенно улучшается; растущее сообщество
Лицензия	Бесплатно для малых доходов (до \$200тыс./год)	Бесплатно, роялти взимается после \$1млн. дохода	Полностью бесплатен, MIT-лицензия
Производительность в WebGL	Оптимизирован	Высокая, но избыточен для казуального проекта	Хорошая, легковесный движок
Совместимость с Яндекс.Играми	Официальный шаблон	Необходима самостоятельная интеграция SDK	Возможно интеграция через JavaScript-мост

Как видно из таблицы, движок Unreal Engine 5 ориентирован преимущественно на высокобюджетные трёхмерные проекты и обладает избыточными для поставленной задачи возможностями, а его сборки для WebGL отличаются значительным размером. Godot 4 представляет собой достойную альтернативу с открытым исходным кодом, однако его сообщество и объём обучающих материалов пока уступают лидерам, а интеграция с Яндекс.Играми требует дополнительных трудозатрат.

Unity, напротив, сочетает полноценную 2D-инструментарий, язык C# с богатой экосистемой библиотек, легкий экспорт в WebGL и официальную поддержку со стороны Яндекс.Игр. Наличие обширной документации и активного сообщества существенно снижает риски при разработке. В совокупности эти факторы позволяют рассматривать Unity в качестве наиболее подходящего кандидата для реализации проекта.

1.6 Обзор дополнительных технологий: архитектурные подходы, SDK и паттерны проектирования

Выбор игрового движка определяет базовый инструментарий, однако не менее важным является предварительное обоснование архитектурных решений и дополнительных технологий, которые будут использованы при реализации. Современный игровой проект, даже будучи казуальным, требует продуманной организации кода, эффективного управления ресурсами и интеграции с сервисами целевой платформы. В данном параграфе рассматриваются подходы и средства, актуальные для будущей разработки.

Компонентно-ориентированная архитектура. Игровой движок Unity, рассмотренный в предыдущем разделе, изначально построен на компонентной модели: каждый игровой объект представляет собой контейнер, на который могут быть добавлены независимые компоненты, инкапсулирующие определённое поведение [4, с.45–48]. Такой подход естественным образом способствует соблюдению принципа единственной ответственности и упрощает повторное использование кода. Для проектируемой игры это означает, что логику различных объектов можно будет распределить среди отдельных компонентов, минимизируя связанность модулей.

Событийно-ориентированная архитектура и паттерн «Наблюдатель» (Observer). Традиционный подход, при котором один компонент напрямую вызывает методы другого, приводит к жёсткой связанности и затрудняет расширение системы. В качестве альтернативы в литературе рекомендуется использование событийного взаимодействия, реализуемого через паттерн «Наблюдатель» [21, с.55–62]. Компоненты не обращаются друг к другу непосредственно, а генерируют события, на которые подписываются заинтересованные обработчики. Для казуальной игры с множеством динамически изменяемых параметров такой подход позволит гибко реагировать на изменения состояния без переписывания существующего кода.

Паттерн «Одиночка» (Singleton). Для глобальных сервисов, управляющих игровым процессом и состоянием игры, хорошим решением будет применять паттерн «Одиночка», он будет гарантировать существование единственного экземпляра класса и предоставлять глобальную точку доступа к нему [21, с.175–180]. Это упрощает взаимодействие между системами, однако требует осторожного использования во избежание неконтролируемого роста зависимостей.

Паттерн «Пул объектов» (Object Pool). Одной из типичных проблем при разработке игр с частым появлением и исчезновением однотипных объектов является избыточная нагрузка на сборщик мусора, приводящая к падению производительности. Паттерн «Пул объектов» предлагает решение: вместо постоянного создания и уничтожения экземпляров они сохраняются в предварительно выделенном пуле и активируются повторно при необходимости [22, с.180–195]. Учитывая, что проектируемая игра предполагает множество различных объектов, применение пула выглядит обоснованным с точки зрения оптимизации.

Конечный автомат (FSM). Для управления поведением объектов, особенно тех, что обладают реакциями на изменение обстановки или сложной логикой, удобным инструментом является модель конечного автомата [21, с.95–110]. Этот подход позволяет описывать поведение как набор состояний с чёткими правилами перехода, что облегчает проектирование сложных сценариев и их последующую отладку.

Yandex Games SDK. Специфика целевой платформы Яндекс.Игры накладывает определённые технические требования, в частности необходимость интеграции Yandex Games SDK. Данный SDK предоставляет JavaScript-интерфейс для доступа к функциям платформы, включая авторизацию, показ рекламы, облачные сохранения и аналитику [14]. Его использование позволит реализовать механизмы монетизации и сбора обратной

связи, рассмотренные в параграфе 1.4, не нарушая общей архитектуры проекта, поскольку вызовы SDK могут быть инкапсулированы в отдельный слой.

Таким образом, анализ дополнительных технологических средств показывает, что сочетание компонентной архитектуры Unity, событийного взаимодействия, паттернов проектирования (Singleton, Object Pool, конечный автомат) и официального SDK Яндекс.Игр формирует необходимый и достаточный набор инструментов для решения поставленной задачи. Конкретные решения по реализации перечисленных подходов будут детализированы в практической части работы.

1.7 Обоснование выбора средств разработки и архитектурного подхода

Результаты анализа позволяют сформулировать итоговый набор инструментов и архитектурных решений для достижения поставленной цели – разработки казуальной компьютерной игры и анализа её продвижения на платформе Яндекс.Игры.

В качестве игрового движка выбран Unity, поскольку он обеспечивает оптимальное соотношение между производительностью WebGL-сборки, удобством двумерной разработки, наличием официального шаблона для Яндекс.Игр и обширной документацией [4][16]. Язык программирования C# позволяет сочетать выразительность с достаточной строгостью типизации.

Архитектурный фундамент проекта составят:

- компонентно-ориентированная модель Unity, обеспечивающая разделение ответственности между модулями [4, с.45–48];
- событийно-ориентированное взаимодействие на основе паттерна «Наблюдатель», снижающее связность систем [21, с. 55–62];
- паттерн «Одиночка» для глобальных сервисов управления игрой [21, с.175–180];
- паттерн «Пул объектов» для оптимизации работы с массовыми однотипными сущностями [22, с.180–195];
- модель конечного автомата для управления сложным поведением объектов [21, с.95–110].

Для интеграции с целевой платформой будет использован Yandex Games SDK, предоставляющий доступ к функциям авторизации, рекламы, облачных сохранений и аналитики [14].

Совокупность выбранных решений обеспечивает ряд преимуществ. Компонентная архитектура и событийное взаимодействие делают код расширяемым и удобным для итеративной доработки, что важно при последующем анализе поведения пользователей и корректировке игровых механик. Пул объектов и конечный автомат позволяют поддерживать стабильную производительность даже при большом количестве сущностей на экране, не снижая отзывчивости управления. Использование официального SDK платформы открывает доступ к встроенным инструментам аналитики. Таким образом, выбранный технологический стек и архитектурный подход формируют фундамент, на котором может быть реализован полноценный игровой продукт, готовый к публикации и последующему изучению его рыночных показателей.

Сформированный набор инструментов и архитектурных решений позволяет перейти к проектированию и реализации игры.

Таким образом, на данном этапе задача по анализу современного состояния рынка и выявлению ключевых факторов успешности продукта, а также задача отбора инструментов, средств и подходов разработки, были выполнены. Полученные результаты служат основой для перехода к практической части, в рамках которой будут решены задачи по проектированию и реализации игры, а также анализу механизмов её продвижения.

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ ПРОДВИЖЕНИЯ ИГРОВОГО ПРИЛОЖЕНИЯ

2.1 Постановка задачи на проектирование и разработку

Результаты, полученные в первой главе, сформировали теоретический фундамент для практической реализации: определены жанровая модель, целевая платформа, базовый инструментарий и ключевые архитектурные решения. В данном параграфе конкретизируются требования к разрабатываемому продукту, переводя их в форму, удобную для непосредственного проектирования.

С учётом выбранной жанровой модели (казуальная игра с элементами rogue-lite) и целевой платформы Яндекс.Игры были сформулированы следующие функциональные требования, описывающие, что именно должно быть реализовано в продукте.

Функциональные требования:

1. Игровой процесс:

- 1.1. Игрок управляет перемещением персонажа с помощью клавиатуры (клавиши WASD или стрелки).
- 1.2. Персонаж автоматически атакует врагов с определённой периодичностью в соответствии с текущими параметрами оружия.
- 1.3. При уничтожении врага выпадает сфера опыта, при подборе которой увеличивается шкала опыта.
- 1.4. При заполнении шкалы опыта повышается уровень персонажа, игра приостанавливается, и на экране отображается панель выбора одного из трёх случайных улучшений.
- 1.5. Игрок может выбрать улучшение, относящееся к одной из трёх категорий: получение нового оружия, усиление характеристик персонажа, усиление текущего оружия.
- 1.6. Одновременно игрок может обладать не более чем четырьмя видами оружия.

- 1.7.Реализовано пять видов оружия с различной механикой атаки: круговая ближняя, конусная ближняя, разлёт снарядов, самонаводящийся снаряд, аура.
2. Враги и волны:
 - 2.1.Враги появляются волнами; каждая волна имеет настраиваемый состав и количество противников.
 - 2.2.Реализовано четыре типа рядовых врагов: преследователь, стрелок, танк, взрывающийся.
 - 2.3.Реализовано три босса, каждый из которых обладает уникальными способностями и визуальными индикаторами атак.
 - 2.4.При гибели врага генерируется событие, EnemySpawner обновляет внутренний счётчик; когда счётчик обнуляется, начинается следующая волна.
 - 2.5.Боссы могут быть включены в волну как обязательные для уничтожения (без их уничтожения волна не считается завершённой).
3. Интерфейс и состояния игры:
 - 3.1.Главное меню содержит кнопки «Начать игру» и «Выход».
 - 3.2.Игровой интерфейс включает полосу здоровья, полосу опыта и счётчик волн.
 - 3.3.Меню паузы (вызов по клавише Escape) содержит кнопки «Продолжить» и «Главное меню».
 - 3.4.Экран окончания игры (Game Over) содержит кнопки «Заново» и «Главное меню».
 - 3.5.Экран победы (Victory) содержит кнопки «Заново» и «Главное меню».
 - 3.6.Камера плавно следует за персонажем.

Нефункциональные требования определяют ограничения, в рамках которых должна разрабатываться и функционировать система. К ним относятся:

- Платформа и технологии

- Разработка ведётся на современном игровом движке Unity с использованием языка C#.
- Конечная сборка выполняется в формат WebGL, обеспечивающий работу в браузере без установки дополнительного программного обеспечения.
- Предусмотрена интеграция Yandex Games SDK для доступа к функциям авторизации, рекламы, аналитики и облачных сохранений.

Архитектура и качество кода:

- Применяется компонентно-ориентированная архитектура, поддерживаемая движком Unity.
- Взаимодействие между модулями организовано по событийно-ориентированной модели с использованием паттерна «Наблюдатель»; прямые вызовы между компонентами сведены к минимуму.
- Для глобальных сервисов (управление игрой, появление врагов) задействован паттерн «Одиночка»; для массовых однотипных объектов (враги, снаряды) – паттерн «Пул объектов»; для управления глобальными состояниями игры и сложным поведением боссов – модель конечного автомата.
- Код оформлен с соблюдением принципа инкапсуляции: публичные поля сведены к минимуму, данные передаются через атрибут SerializeField и свойства, что снижает связанность и облегчает рефакторинг.
- Архитектура допускает добавление новых типов врагов, оружия и улучшений без модификации существующих модулей.

Производительность:

- Частота кадров должна оставаться стабильной (не ниже 30 FPS) при одновременном присутствии на сцене не менее 50 врагов и активной стрельбе.

- Благодаря использованию пулов объектов исключается прогрессирующее накопление лишних игровых объектов и снижение производительности при длительных игровых сессиях.

Надёжность:

- Приложение должно корректно обрабатывать все предусмотренные состояния (главное меню, игра, пауза, выбор улучшения, победа, поражение) и обеспечивать переходы между ними без потери данных.
- Стандартный сценарий игры (от запуска до завершения игрового цикла) не должен приводить к критическим ошибкам, вызывающим аварийное завершение приложения.

Сформулированные требования образуют достаточную основу для перехода к проектированию архитектуры приложения.

2.2 Архитектура игрового приложения

Проектирование архитектуры игрового приложения базируется на возможностях движка Unity и выбранном наборе паттернов проектирования. Основопологающим принципом является компонентно-ориентированный подход, при котором каждый игровой объект представляет собой контейнер, на который навешиваются независимые компоненты, инкапсулирующие конкретную функциональность. Такой подход поддерживается Unity и позволяет добиться слабой связанности между модулями.

Вся кодовая база проекта структурирована в виде набора скриптов на языке C#, размещённых в папке Assets/Scripts. Логически связанные скрипты сгруппированы по подпапкам: Player (компоненты персонажа), Enemy (общие вражеские компоненты и спаунер), Enemy/Bosses (боссы), Enemy/CommonEnemy (поведение рядовых противников), Attacks (стратегии атак оружия), Weapon (данные оружия, менеджер улучшений). Такая организация облегчает навигацию по проекту и расширение его функциональности.

Центральным элементом, связывающим разрозненные модули, является событийно-ориентированная система. Она реализована через статический класс `GameEvents`, который содержит набор событий и статические методы для их вызова. Компоненты подписываются на интересующие их события в момент активации и отписываются при деактивации. Так, например, изменение здоровья персонажа порождает событие `OnPlayerHealthChanged`, которое получает UI-слой и обновляет полосу здоровья. Смерть врага порождает событие `OnEnemyDeath`, на которое реагирует `EnemySpawner`, корректируя счётчик оставшихся противников. Эта модель исключает необходимость в прямых ссылках между компонентами и упрощает добавление новых подписчиков.

Глобальное управление жизненным циклом приложения осуществляет singleton `GameManager`. Он хранит текущее состояние игры (набор именованных целочисленных констант `GameState`), предоставляет методы для смены состояния и управляет временной шкалой (`Time.timeScale`) в зависимости от состояния. Конечный автомат состояний включает: `MainMenu`, `Playing`, `Paused`, `UpgradeSelection`, `GameOver`, `Victory`. Переходы между состояниями инициируются либо пользовательскими действиями (нажатие `Escape`, выбор кнопки в UI), либо игровыми событиями (смерть персонажа, завершение всех волн). Каждый переход сопровождается вызовом соответствующего события, что позволяет, например, UI-слою показывать или скрывать нужные панели исключительно по факту изменения состояния.

За управление врагами отвечает синглтон `EnemySpawner`. Он содержит конфигурацию волн (ассет `WaveSet`), ссылки на пулы объектов (`ObjectPool`) для разных типов врагов и генерирует события `OnWaveStarted` и `OnAllWavesCompleted`. Для каждого типа врагов создан отдельный пул, что позволяет избежать частого создания и уничтожения экземпляров и поддерживать стабильную производительность при большом количестве противников на сцене.

Компоненты, отвечающие за поведение персонажа, расположены в папке `Player` и включают `PlayerStats` (характеристики), `PlayerMovement` (перемещение),

PlayerHealth (получение урона и лечение), PlayerDirection (определение направления взгляда) и PlayerWeaponManager (управление оружием). Последний хранит коллекцию активных экземпляров RuntimeWeapon и вызывает их метод PerformAttack, передавая координаты и направление атаки.

Сама система оружия построена на паттерне «Стратегия»: каждое оружие обладает ссылкой на объект, реализующий интерфейс IAttackBehaviour, что позволяет единообразно обрабатывать атаки, различающиеся по механикам (круговая, конусная, снаряды, аура). Данные конкретного оружия хранятся в ScriptableObject-ассетах (WeaponData), которые содержат не только числовые параметры (урон, длительность перезарядки, дальность), но и указание на тип атаки и соответствующий префаб визуального эффекта. Такой подход упрощает балансировку и добавление новых видов оружия без изменения кода.

Поведение врагов также организовано по компонентному принципу. Базовые параметры (здоровье, награда опытом, контактный урон) вынесены в EnemyStats; движение – в EnemyMovement; урон при контакте с игроком – в EnemyContactDamage. Специфическая логика (стрельба, взрыв) реализована отдельными компонентами (EnemyShooter, EnemyExploder), что позволяет комбинировать их в разных комбинациях. Для боссов применена модель конечного автомата, инкапсулированная в классах-наследниках общего базового класса MiniBoss. Каждый босс управляет собственным набором состояний (например, преследование, скольжение, подготовка рывка, подготовка конусной атаки), переключение между которыми осуществляется по условиям, заданным в коде.

Улучшения параметров персонажа и оружия во время игровой сессии реализована через singleton UpgradeManager. При повышении уровня (событие OnPlayerLevelUp) игра переходит в состояние UpgradeSelection, а UpgradeManager генерирует список из трёх случайных улучшений, отфильтрованных по заданным заранее правилам. Панель выбора (UpgradePanelUI) отображает эти улучшения, а после клика игрока вызывает метод ApplyUpgrade, модифицирующий либо характеристики персонажа

(PlayerStats), либо параметры конкретного RuntimeWeapon, либо добавляющий новое оружие в коллекцию PlayerWeaponManager.

Для наглядного представления организации кодовой базы на рисунке 1 приведена файловая структура проекта, отражающая разделение скриптов по модулям.



Рисунок 1

Таким образом, архитектура приложения опирается на компонентный подход, событийное взаимодействие и набор проверенных паттернов проектирования, что обеспечивает гибкость, расширяемость и соответствие нефункциональным требованиям.

2.3 Реализация ключевых компонентов

Реализация ключевых компонентов выполнялась на языке C# в среде Unity. Ниже рассмотрены основные подсистемы, образующие основу и ключевые аспекты проекта.

Персонаж. Игровой объект «Player» является контейнером для набора независимых компонентов, каждый из которых отвечает за отдельный аспект поведения. Компонент PlayerStats хранит текущие и максимальные значения здоровья, опыта, уровня, а также дополнительные параметры (скорость передвижения, множитель опыта). Изменение здоровья и опыта сопровождается вызовом статических событий OnPlayerHealthChanged и OnPlayerExperienceChanged, на которые подписан UI-слой. Достижение порога опыта приводит к вызову OnPlayerLevelUp, что инициирует переход игры в состояние выбора улучшения.

Перемещение реализовано компонентом PlayerMovement, который считывает ввод с клавиатуры и в методе FixedUpdate задаёт линейную скорость физическому телу (Rigidbody2D). Компонент PlayerDirection на основе вектора ввода определяет текущее направление «взгляда» персонажа, необходимое для корректной ориентации визуальных эффектов и атак, требующих указания направления (конус, разлёт снарядов). Контроль здоровья и получение урона вынесены в PlayerHealth, что позволяет при необходимости добавлять логику неуязвимости или визуальную обратную связь, не затрагивая остальные модули.

Управление оружием возложено на PlayerWeaponManager. Он хранит список активных экземпляров RuntimeWeapon (максимум четыре) и в каждом кадре вызывает метод PerformAttack, передавая позицию и направление атаки. Визуальный эффект каждой атаки порождается через callback (обратный вызов),

что позволяет разным типам оружия использовать собственные префабы эффектов. Для ауры создаётся постоянный дочерний объект, позиция и масштаб которого обновляются синхронно с персонажем.

Система оружия. Каждое оружие описывается ассетом типа `WeaponData` (наследник `ScriptableObject`), в котором заданы базовые параметры: урон, кулдаун, дальность, тип атаки (`Melee`, `Projectile`, `Aura`), а также специфические для подтипа поля (угол конуса, количество снарядов, интервал тиков ауры). Во время игры эти данные копируются в экземпляр `RuntimeWeapon`, который дополнительно хранит текущие модифицированные параметры и счётчик времени перезарядки.

Сама логика атаки инкапсулирована в классах, реализующих интерфейс `IAttackBehaviour`. Стратегия выбирается в конструкторе `RuntimeWeapon` на основе типа и подтипа оружия. В проекте реализованы пять стратегий: `CircleMeleeAttack` (урон и отталкивание всех врагов в радиусе), `ConeMeleeAttack` (аналогично, но с фильтрацией по углу относительно направления взгляда и без функции отталкивания), `SpreadProjectileAttack` (залп из нескольких снарядов в заданном секторе), `HomingProjectileAttack` (один снаряд, направляющийся к ближайшему врагу), `AuraAttack` (периодический урон всем врагам в радиусе без отталкивания). Все AoE-атаки (`Circle`, `Cone`, `Aura`) для обнаружения целей используют метод `Physics2D.OverlapCircleAll`, что гарантирует попадание при касании коллайдеров и не требует вычисления расстояния между центрами.

Снаряды игрока (`PlayerProjectile`) также используют пул объектов. При инициализации снаряду передаются направление, урон, скорость, время жизни и ссылка на пул, в который он должен вернуться после попадания или выхода за пределы экрана. Это исключает накопление мусора при частой стрельбе.

Враги. Для представления врагов используется компонентная модель, аналогичная модели персонажа. Общими для всех противников являются компоненты `EnemyStats` (здоровье, урон при контакте, награда опытом), `EnemyMovement` (движение к игроку с возможностью остановки и отталкивания) и `EnemyContactDamage` (нанесение урона персонажу при касании).

Специфическое поведение добавляется отдельными компонентами: `EnemyShooter` для стрелков (остановка на расстоянии атаки, периодическая стрельба снарядами) и `EnemyExploder` для взрывающихся врагов (остановка при приближении к игроку или при смерти, задержка, взрыв с уроном по области). Танк отличается лишь увеличенными значениями здоровья, контактного урона и массы, поэтому не требует дополнительного скрипта. Такая декомпозиция позволяет комбинировать компоненты и создавать новые типы врагов без дублирования кода.

Боссы. Для реализации трёх боссов применена модель конечного автомата, инкапсулированная в наследниках общего базового класса `MiniBoss`. Каждый босс управляет собственным набором состояний, переходы между которыми задаются условиями (расстояние до игрока, уровень здоровья, таймеры). Например, босс «Берсерк» при снижении здоровья ниже 50% переходит в состояние ярости, увеличивая скорость и меняя логику движения: вблизи игрока он фиксирует направление и скользит по прямой, не корректируя курс, затем замедляется до остановки и снова начинает преследование. Босс «Снайпер» использует состояния преследования и стрельбы с тройным залпом, а также периодически телепортируется. Босс «Танк» переключается между рывком (прямоугольная область, прицеливание, разгон) и конусной атакой (отрисовка конуса, задержка, урон). Все состояния сопровождаются визуальными индикаторами, которые создаются и уничтожаются динамически в соответствии с текущим состоянием.

Волновая система и спаунер. Управление появлением врагов возложено на синглтон `EnemySpawner`. Конфигурация всех волн хранится в ассете `WaveSet`, каждая запись которого задаёт количество врагов, интервал между появлениями и состав (опциональный список обязательных противников и список врагов со случайными весами). При старте волны сначала появляются обязательные враги (обычно боссы), затем оставшиеся места заполняются случайными

противниками согласно весам. Спаунер использует предварительно созданные пулы объектов (ObjectPool) для каждого префаба врага, что исключает накладные расходы на создание новых экземпляров во время игры. Позиция для появления выбирается за пределами видимой области камеры с проверкой отсутствия коллизий со стенами, что гарантирует появление врагов на допустимой территории. При смерти врага генерируется событие OnEnemyDeath, спаунер удаляет его из списка активных, при необходимости создаёт сферу опыта и возвращает объект в пул. Завершение волны фиксируется при обнулении счётчика оставшихся врагов и отсутствии активных противников. После последней волны вызывается событие OnAllWavesCompleted, переводящее игру в состояние победы.

Пул объектов. Компонент ObjectPool реализует классический паттерн «Пул объектов». При инициализации создаётся заданное количество экземпляров указанного префаба, которые помещаются в очередь. Метод Get извлекает объект, активирует его и размещает в указанной точке; если очередь пуста, динамически создаётся дополнительный экземпляр. Метод ReturnToPool деактивирует объект и возвращает в очередь. Пул используется для всех типов врагов, снарядов игрока, сфер опыта и вражеских снарядов. Благодаря этому количество операций создания и уничтожения объектов сведено к минимуму, что положительно сказывается на стабильности частоты кадров, особенно в сценах с большим количеством врагов.

Прокачка и UI выбора улучшений. Система прокачки задействует singleton UpgradeManager, который подписывается на событие OnPlayerLevelUp. При его срабатывании игра переходит в состояние UpgradeSelection, и менеджер генерирует список из трёх случайных улучшений, отбираемых из общего пула всех ассетов WeaponUpgradeData. Фильтрация учитывает, какие оружия уже есть у игрока, максимальное количество покупок для уникальных улучшений и категорию улучшения (новое оружие, усиление персонажа, усиление

конкретного оружия). Сгенерированный набор передаётся панели UpgradePanelUI, которая создаёт кнопки с названиями, описаниями и иконками улучшений. При клике игрока UpgradeManager применяет выбранный апгрейд: модифицирует соответствующие поля в PlayerStats либо в RuntimeWeapon, либо добавляет новое оружие в PlayerWeaponManager. После этого игра возвращается в состояние Playing.

Управление состояниями игры. Центральным диспетчером является синглтон GameManager, реализующий конечный автомат с шестью состояниями: главное меню, игра, пауза, выбор улучшения, поражение, победа. Смена состояния осуществляется методом SetState, который управляет временной шкалой и вызывает событие OnGameStateChanged. Это событие, в свою очередь, позволяет UI-слою показывать или скрывать соответствующие панели без явных ссылок на GameManager. Переходы инициируются пользовательскими действиями (кнопки меню, клавиша Escape) либо игровыми событиями (смерть персонажа, завершение всех волн, повышение уровня). Такая организация гарантирует корректное поведение игры при любых сценариях и упрощает добавление новых состояний в будущем.

Таким образом, реализованные компоненты в совокупности образуют законченный игровой продукт, соответствующий функциональным и нефункциональным требованиям, и подтверждают жизнеспособность выбранных архитектурных решений.

2.4 Тестирование и оптимизация

Разработанный продукт был подвергнут комплексной проверке, включавшей как функциональное тестирование силами разработчика в процессе реализации, так и привлечение внешних пользователей для оценки геймплейных качеств и стабильности приложения. Такой подход позволил не только выявить

и устранить скрытые дефекты, но и собрать обратную связь, важную для оценки соответствия продукта ожиданиям целевой аудитории.

Для проведения пользовательского тестирования была сформирована группа из шести человек в возрасте от 18 до 26 лет, что практически полностью перекрывает демографический профиль, характерный для аудитории казуальных игр на платформе Яндекс.Игры. При отборе участников соблюдался принцип разнородности: большинство тестировщиков обладали значительным игровым опытом, в особенности в проектах, близких к жанру *rogue-lite*, однако в группу были включены и пользователи с минимальным игровым стажем. Это дало возможность оценить, насколько интуитивным является управление и насколько быстро осваиваются механики без предварительной подготовки. Важно отметить, что подавляющее большинство участников не имело профильных знаний в сфере информационных технологий или разработки игр, что исключало профессионально обусловленную предвзятость при оценке продукта.

Каждому тестировщику предлагалось пройти игру от начального меню до завершения нескольких волн, включая встречи с боссами и взаимодействие с системой выбора улучшений. Особое внимание уделялось следующим показателям: понятность интерфейса, отзывчивость управления, читаемость визуальных индикаторов атак, общее впечатление от игрового процесса, а также наличие ошибок, приводящих к некорректному поведению или аварийному завершению приложения.

Результаты тестирования показали, что все участники успешно справились с поставленной задачей, а средняя продолжительность игровой сессии превысила десять минут. Данный показатель важен с точки зрения требований платформы Яндекс.Игры, для которой минимальная длительность сессии является одним из критериев, влияющих на прохождение модерации продуктом. Управление было охарактеризовано как интуитивно понятное, интерфейс не вызывал затруднений даже у пользователей с минимальным игровым опытом.

Положительную оценку получило поведение мини-боссов. Игроки отметили, что, не являясь уникальными с точки зрения геймдизайна, боссы

демонстрируют качественно реализованные атаки с понятными визуальными индикаторами, что делает сражения с ними тактически осмысленными. В целом проект был воспринят как имеющий надёжный фундамент для дальнейшего расширения. Вместе с тем прозвучали пожелания о наполнении игры дополнительным контентом: большим разнообразием оружия, персонажей и механик, позволяющих строить более глубокие и уникальные конфигурации (билды). Эти замечания носят перспективный характер и не влияют на оценку текущей реализации в рамках поставленных задач.

В ходе тестирования был выявлен ряд дефектов, подлежащих немедленному исправлению. Наиболее существенным из них оказалась ошибка, при которой враги, извлекаемые из пула объектов повторно, могли наследовать некорректное состояние от предыдущего цикла использования (например, оставаться в режиме стрельбы или подготовки взрыва без возможности движения). Причина заключалась в неполной инициализации флагов и таймеров при активации объекта. Проблема была устранена путём добавления в компоненты врагов метода `OnEnable`, в котором все динамические поля принудительно сбрасываются в исходные значения. Кроме того, на основании субъективных отзывов была доработана механика скольжения босса «Берсерк»: в первоначальной версии босс продолжал доворачивать в сторону игрока во время скольжения, что противоречило задуманной иллюзии инерционного движения и снижало возможности для уклонения. Логика была изменена таким образом, чтобы на фазе скольжения направление фиксировалось в момент начала манёвра и не корректировалось вплоть до остановки. Все исправления были внесены в код и верифицированы повторными запусками.

Помимо функционального и пользовательского тестирования, была проведена оценка производительности в условиях высокой нагрузки. Стресс-тест выполнялся локально на персональном компьютере разработчика в среде Unity Editor. На сцену одновременно помещались три босса с активным поведением (каждый из которых порождает визуальные индикаторы и обрабатывает логику своих состояний) и более ста рядовых врагов, что примерно

в два раза превышало расчётную нагрузку для стандартной волны. Несмотря на столь высокую концентрацию объектов, частота кадров оставалась в допустимых пределах (значительно выше 30 FPS), физический движок корректно обрабатывал коллизии и отталкивания, а поведение врагов не демонстрировало признаков деградации. Следует оговориться, что тестирование проводилось на устройстве с производительным аппаратным обеспечением, однако архитектурные решения, принятые при разработке (в первую очередь повсеместное использование пулов объектов), позволяют ожидать сопоставимых результатов и на менее мощных конфигурациях.

Таким образом, проведённое тестирование подтвердило, что разработанный продукт удовлетворяет всем предъявленным функциональным и нефункциональным требованиям, демонстрирует стабильную работу в пределах расчётных нагрузок и положительно воспринимается представителями целевой аудитории. Выявленные на этапе тестирования недостатки были своевременно устранены, что обеспечило качество итоговой реализации, достаточное для перехода к анализу механизмов продвижения на целевой платформе.

2.5 Анализ механизмов продвижения на платформе Яндекс.Игры

Настоящая работа предполагает анализ инструментов продвижения, встроенных в платформу Яндекс.Игры, и оценку их потенциальной эффективности применительно к разработанному продукту. Анализ выполняется на основе изучения официальной документации платформы и открытых источников, а также с учётом архитектурных и геймдизайнерских решений, заложенных в игру.

Публикация игры на Яндекс.Играх требует выполнения ряда обязательных требований, которые можно условно разделить на технические, пользовательские и контентные. Разработанный продукт в текущем состоянии удовлетворяет большинству из них: игра представляет собой WebGL-сборку, не требующую установки, в неё может быть интегрирован Yandex Games SDK,

управление осуществляется клавиатурой (для десктопной версии), а длительность стандартной сессии превышает 10 минут, что соответствует минимальному порогу, установленному платформой. Таким образом, базовые условия для размещения и начала продвижения соблюдены.

Ключевым механизмом органического продвижения в каталоге Яндекс.Игр является ранжирование, основанное на алгоритмах машинного обучения. Система оценивает такие метрики, как количество игровых сессий, их продолжительность, рейтинг игры, количество возвращений игроков и доход. Разработанный продукт обладает характеристиками, способными положительно влиять на эти показатели. Выбранная жанровая модель (короткие сессии, необратимая потеря прогресса, внутриигровая прогрессия через выбор улучшений) стимулирует повторные прохождения и увеличивает как количество сессий, так и их суммарную продолжительность. Нарастающая сложность волн и наличие нескольких типов врагов с уникальным поведением поддерживают интерес на протяжении длительного времени, что способствует удержанию игроков и повышению вероятности возвращения в игру.

Особое значение для ранжирования имеет рейтинг игры, который формируется на основе отзывов пользователей. Проведённое пользовательское тестирование показало, что продукт положительно воспринимается целевой аудиторией, а выявленные недостатки были устранены. Это позволяет ожидать достаточно высокого начального рейтинга, что, в свою очередь, улучшит позиции игры в каталоге.

Другим важным инструментом органического продвижения является фичеринг – попадание в тематические подборки. Для этого платформа оценивает «заполненность карточки», то есть наличие качественных промоматериалов, описаний, иконок и обложек. На данном этапе проект располагает необходимым визуальным контентом (скриншоты геймплея, демонстрирующие основные механики, читаемый интерфейс), что облегчает подготовку материалов для карточки. Возможность проведения А/В-тестов иконок и обложек, предоставляемая платформой, позволит в дальнейшем экспериментально

подобрать наиболее привлекательные креативы и повысить конверсию из показов в запуски.

Платное продвижение, реализуемое через интеграцию с Яндекс.Директом, даёт возможность ускорить приток аудитории на начальном этапе. Выбор стратегии «Максимум целевых действий» с ориентацией на длительность сессии позволит привлечь именно ту аудиторию, которая склонна к продолжительному взаимодействию с игрой, что согласуется с её механиками. При этом архитектурные решения, в частности использование пулов объектов и оптимизация производительности, гарантируют, что даже при резком увеличении числа игроков серверная нагрузка (в части, зависящей от клиентской сборки) останется в допустимых пределах.

Встроенные инструменты монетизации (реклама и внутриигровые покупки) также косвенно влияют на продвижение, поскольку доход является одной из метрик ранжирования. Разработанный продукт предусматривает возможность интеграции рекламных блоков в логических паузах (например, после гибели персонажа), что не нарушает игровой процесс и соответствует требованиям платформы.

На основе проведённого анализа можно сформулировать следующие рекомендации по улучшению видимости продукта:

- обеспечить полную локализацию интерфейса на английский язык, что расширит потенциальную аудиторию примерно на 40% (согласно данным платформы о доле международного трафика);
- подготовить несколько вариантов иконок и обложек для запуска A/B-тестов с целью повышения CTR в каталоге;
- поддерживать регулярный цикл обновлений, добавляя новые виды оружия и врагов, что положительно скажется на удержании и возвращаемости;
- рассмотреть возможность проведения промоакций (например, сезонных событий), которые платформа активно поддерживает и продвигает в специальном разделе.

Разработанный продукт обладает набором характеристик, позволяющих эффективно использовать как органические, так и платные инструменты продвижения платформы Яндекс.Игры. Заложенные в него механики располагают к достижению высоких значений ключевых метрик, а архитектурная гибкость облегчает дальнейшую адаптацию под меняющиеся требования платформы и предпочтения аудитории.

2.6 Итоги практической реализации и анализа

Практическая реализация подтвердила работоспособность выбранного технологического стека и архитектурных подходов. Событийно-ориентированная модель, компонентная структура и применение паттернов проектирования позволили создать модульную и расширяемую систему, в которой добавление новых механик (оружия, типов врагов, улучшений) не требует переписывания существующего кода. Это особенно важно в контексте дальнейшего развития продукта, необходимость которого была отмечена как тестировщиками, так и в ходе анализа инструментов продвижения.

Отдельного упоминания заслуживает баланс между казуальной доступностью и тактической глубиной, достигнутый за счёт комбинации простого управления, автоматических атак и многоуровневой системы выбора улучшений. Тестирование с участием представителей целевой аудитории показало, что игра интуитивно понятна новичкам, но при этом способна удерживать внимание более искушённых игроков на протяжении десятков минут. Это свидетельствует о верно выбранной геймдизайнерской формуле и создаёт предпосылки для высоких показателей удержания – ключевой метрики платформы Яндекс.Игры.

Анализ механизмов продвижения позволил установить, что разработанный продукт обладает характеристиками, востребованными платформой. Короткая игровая сессия, нарастающая сложность, стимулирование повторных прохождений и наличие визуально-читаемых механик прямо коррелируют с

теми метриками, которые учитываются при ранжировании и фичеринге. Сформулированные рекомендации по локализации, А/В-тестированию креативов и поддержке регулярных обновлений дают практический вектор для дальнейшего развития проекта после его публикации.

Таким образом, в рамках второй главы полностью решены поставленные задачи. Разработанный продукт демонстрирует работоспособность и соответствует функциональным и нефункциональным требованиям, а проведённый анализ даёт обоснованное представление о механизмах его продвижения на платформе Яндекс.Игры. Совокупность полученных результатов подтверждает достижение цели работы.

ЗАКЛЮЧЕНИЕ

Представленная выпускная квалификационная работа посвящена разработке казуальной компьютерной игры и анализу её продвижения на платформе Яндекс.Игры. В ходе выполнения работы поставленная цель была достигнута, а все сформулированные задачи – решены.

В рамках задачи по анализу современного состояния рынка казуальных компьютерных игр было установлено, что данный сегмент сохраняет устойчивый рост, а браузерные платформы играют в нём всё более заметную роль. Выявлены ключевые особенности казуальных игр – простота правил, короткая сессия, интуитивное управление, – а также определена перспективность гибридных моделей, сочетающих казуальную доступность с механиками *rogue-lite*. Применительно к платформе Яндекс.Игры были выделены факторы, влияющие на успешность продукта: соответствие техническим требованиям, ориентация на мобильную и десктопную аудиторию, а также способность игры демонстрировать высокие показатели удержания и продолжительности сессий.

В рамках задачи по отбору инструментальных средств разработки были проанализированы современные игровые движки, архитектурные подходы и дополнительные SDK. На основе сравнительного анализа в качестве основного инструмента выбран Unity, обосновано применение компонентно-ориентированной архитектуры, событийно-ориентированного взаимодействия, а также паттернов проектирования – «Наблюдатель», «Одиночка», «Пул объектов» и конечный автомат. Дополнительно определена необходимость интеграции Yandex Games SDK для доступа к функциям платформы.

В рамках задачи по проектированию архитектуры и реализации игрового цикла была спроектирована модульная структура приложения, разработаны все ключевые компоненты и обеспечено их взаимодействие через статические события. Созданный продукт включает пять видов оружия с различной механикой атаки, четыре типа рядовых врагов, трёх боссов с уникальным

поведением, волновую систему, механику прокачки с выбором улучшений и полный набор игровых состояний – от главного меню до экранов победы и поражения. Проведённое тестирование с участием представителей целевой аудитории подтвердило работоспособность продукта, его интуитивную понятность и соответствие жанровым ожиданиям, а оптимизационные решения, включая пул объектов, обеспечили стабильную производительность.

В рамках задачи по анализу механизмов продвижения на платформе Яндекс.Игры были детально рассмотрены инструменты органического и платного продвижения: система ранжирования, фичеринг, A/B-тестирование креативов, промоакции и рекламные кампании. Оценена их потенциальная эффективность применительно к разработанному продукту и показано, что заложенные в игру механики способствуют достижению высоких значений ключевых метрик платформы – удержания, продолжительности сессий и возвращаемости. Сформулированы практические рекомендации по повышению видимости продукта, включающие локализацию на английский язык, подготовку креативов для A/B-тестов и поддержку регулярных обновлений.

Все поставленные задачи решены, цель работы достигнута. Разработанный продукт представляет собой законченную казуальную игру, готовую к публикации на платформе Яндекс.Игры, а проведённый анализ даёт разработчику инструментарий для осознанного управления её продвижением.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Adams E. Fundamentals of Game Design. – 3rd ed. – Berkeley: New Riders, 2014. – 576 p.
2. Newzoo. Global Games Market Report 2024. – [Электронный ресурс]. – Режим доступа: <https://newzoo.com/resources/trend-reports/newzoos-global-games-market-report-2024-free-version> (дата обращения: 19.05.2026).
3. Яндекс.Игры. Официальный сайт для разработчиков. – [Электронный ресурс]. – Режим доступа: <https://yandex.ru/dev/games/doc/ru/> (дата обращения: 19.05.2026).
4. Borromeo N. A. Hands-On Unity 2022 Game Development: Learn to use the latest Unity 2022 features to create your first video game. – Birmingham : Packt Publishing, 2022. – 712 p.
5. Doran J. P. Unity 2022 Mobile Game Development: Build and publish engaging games for Android and iOS. – Birmingham : Packt Publishing, 2023. – 480 p.
6. Santos R. Unity 2022 by Example. – Birmingham : Packt Publishing, 2022. – 404 p.
7. Thorn A. Game Development Principles with Unity. – Boston : Cengage Learning, 2022. – 560 p.
8. Rogers S. Level Up! The Guide to Great Video Game Design. – 3rd ed. – Hoboken : Wiley, 2022. – 600 p.
9. Fullerton T. Game Design Workshop: A Playcentric Approach to Creating Innovative Games. – 5th ed. – Boca Raton : CRC Press, 2024. – 522 p.
10. Schell J. The Art of Game Design: A Book of Lenses. – 4th ed. – Boca Raton : CRC Press, 2023. – 640 p.
11. Яндекс.Игры. Аналитика игр на площадке. – [Электронный ресурс]. – Режим доступа: <https://yandex.ru/dev/games/doc/ru/concepts/analytics> (дата обращения: 21.05.2026).

12. Измайлова А. А., Будрин А. Г., Солдатова А. В. Анализ методов продвижения мобильных приложений в сфере gamedev // Экономика. Право. Инновации. – 2023. – № 2. – С. 29–37.
13. Craddock D. L. Dungeon Hacks: How NetHack, Angband, and Other Roguelikes Changed the Course of Video Games. – Boca Raton : CRC Press, 2021. – 172 p.
14. Яндекс.Игры. Документация Yandex Games SDK. – [Электронный ресурс]. – Режим доступа: <https://yandex.ru/dev/games/doc/ru/sdk> (дата обращения: 21.05.2026).
15. Яндекс.Игры. Требования к публикации игр. – [Электронный ресурс]. – Режим доступа: <https://yandex.ru/dev/games/doc/ru/concepts/requirements> (дата обращения: 22.05.2026).
16. Unity Technologies. Unity Manual 2022 LTS. – San Francisco : Unity Technologies, 2022.
17. Unity Technologies. Unity Documentation : официальный сайт. – [Электронный ресурс]. – Режим доступа: docs.unity3d.com (дата обращения: 20.05.2026).
18. Голенок А. А. Способы продвижения современных мобильных игр // Информационные системы и технологии. Информационная безопасность. – 2021. – № 1. – С. 52–60.
19. Андреев Н. Ю., Архипов В. В., Васильев А. А., Печатнова Ю. В. Правовые аспекты способов монетизации прав в индустрии компьютерных игр // Правоприменение. – 2023. – Т. 7, № 1. – С. 83–92.
20. Яндекс.Игры. Монетизация игр. – [Электронный ресурс]. – Режим доступа: <https://yandex.ru/dev/games/doc/ru/services/about-monetization> (дата обращения: 22.05.2026).
21. Nystrom R. Game Programming Patterns. – Genever Benning, 2021. – 354 p.

22. Aversa A., Dickinson C. Unity Game Optimization: Enhance and extend the performance of all aspects of your Unity games. – 3rd ed. – Birmingham : Packt Publishing, 2024. – 456 p.